Response
From X990 POS
Request
To X990
Print slip
X990 POS
Accounting Software
PC based POS

1. Contents

2. Document history

Initial Draft	Date	Author	changes
V0	1/3/2023	PCNC	Initial draft
V1	28/03/2023	Anas	1. Added missing fields in the Void request table.at page 18. 2. Added request header and length. At page 11
V2	11/05/2023	Anas	Added notes for encryption/decryption with AES & RSA. Indicated the padding mode in RSA, and cipher mode in AES
V3	24/05/2023	Anas	Clarified AES key size, 256-bits
V4	09/07/2023	Anas	1. Added batch time transaction. 2. Added trans type and voided flag to trans details.
V5	23/12/2024	Anas	Added two new mandatory fields to the request of cancel (void/return) sale trans. 1. datim: string with format **yyMMddHHmmss**, sample: 241223152000 2. authCode: string

3. Solution Introduction:

The purpose of this project is to open a secure connection to the existing BOP POS machines X990 which will allow any 3rd party system (PC Based POS, Kiosk , self-service kiosks) to use the X990 functionality as a PINPad .when required , the MSU will setup the X990 in a way that it will act as PINPAD. The 3rd party companies, can communicate to the X990 using a standard IP and standard API's, which will allow all 3rd party companies to integrate with the X990 POS machine using the listed APIs in the technical section below .

This solution will allow the support for BOP group payments using :

1- Any Debit / Credit card local and international.

2- QR code payments for BOP Banke / AIB Banke and PalPay Mahfazati in addition to any other bank that will join the BOP group QR payment.

For the QR new APIs has been added to allow the 3rd party to accept the payments using BOP QR methods.

4. Solution Features:

1. Accept all payment methods, including EMV and NFC/Contactless Cards and QR codes.

2. Improve the customer experience with best-in-class features, such as color screen, animations and sound alerts

3. Simplify solution integration, operation and scalability.

5. Requirements Specifications

3rd Party software integration (Any merchant system, developed on any platform).

• The 3rd party to be able to communicate with the POS terminal and send specific request listed below and get specific results as per the table below using standard TCP/IP protocol the 3rd party system and the X990 terminal.

• The 3rd Party will receive in the response the transaction processing confirmation, status, and details without compromising the security.

• The X990 should have the capability to print or not to print the payment slip after transaction processing. If printed the slip will show success/failed payment.

6. Supported transactions and X990 keypad activity:

- **INIT** – Exchanging RSA keys for encryption
- **SALE** - Sale Transaction
- **SALE_CB** - Sale with Cash-back Transaction
- **LOAN** - Loan Transaction
- **VOID** - Void Transaction
- **RETURN** - Return / Refund Transaction
- **QUERY** - Query Transaction
- **BATCH** – close current terminal batch
- **QR Payment transaction.**
- **BATCH_TIME –** Set batch time

7. Transaction Flow:

A. Customer/Merchant make a sale transaction on his PC/Tablet, then click on Pay with X990 Pin pad.

B. X990 Will Receive the amount to be paid and display it on the terminal.

C. Customer Tap/Insert card or Read QR to confirm the payment.

D. After the payment is done by the customer, a confirmation will be sent from the POS terminal to the requester 3rd party application including the payment transaction details.

E. If the X990 is configured to print the slip, the POS machine will display a choice for the client to print or not. in addition, the 3rd party can send the print slip flag with the request indicating to print the slip on the X990 or not.

8. Printing terminal slip-Sale Transaction

Please note that you need to print this slip after printing the cash register slip . the idea is that the client will get the list of the products ,the total that has been paid by Credit/ Debit /QR and list the below slip in the below layout. Please note that the slip title should indicate the word “original” . it is also required to store the slip information for reprinting a copy of the slip whenever required. The second slip should be Titled with the word “COPY” only 1 Original slip can be printed.

بنك فلسطين
BANK OF PALESTINE

TESTTTT
GAZA
Bethlehem

POS Code: 11252665
Outlet No: 456000456

**** ORIGINAL ****
CUSTOMER SLIP
SALE

2023-03-23 13:44:29
Card #: ****5836
Exp. Dt: **/**
Seq. No: 000334
Auth. Code: 758895
Card Type: VISA BOP DB
Card Entry: CTLS
AID: A0000000031010

GOODS/SERVICES RECEIVED

APPROVED 250. 00 ILS

PIN:xxxxxxxx
/

TVR:0000000000 TSI:0000
CID:80 Terminal Type:22

Thank You
No RETURN/CANCELLATION
Please, Keep the Receipt

**** ORIGINAL ****
*** Bank of Palestine PLC and PALPAY ***
Pal_PAY 05.00.17R - Mar 22 2023 09:35:43

بنك فلسطين
BANK OF PALESTINE

merch
TESTTTT
GAZA
pos_code
Bethlehem
POS Code: 11252665
Outlet No: 456000456
outlet

**** ORIGINAL ****
CUSTOMER SLIP

SALE

datim
2023-03-23 12:02:14
Card #: ****3419
Exp. Dt: **/**
Seq. No: 000317
seq
Auth. Code: 014337
stan
Card Type: MasterCard
type
Card Entry: ICC
AID: A0000000041010

dcc
9278 JPY
250.00 ILS

GOODS/SERVICES RECEIVED

APPROVED

response_code
exchange rate
amt

Sale Amount: 250.00 ILS
Exchange Rate 1 ILS= 36.03 JPY
Currency Conversion Fee: 3.00%
dcc trans currency
Trans Currency: JPY
Trans Amount: 9278 JPY

I have been offered a choice of currencies and agreed to pay in YEN(JPY)

Dynamic Currency Conversion (DCC) is offered by TESTTTT

Sign: ____________
RISHMAWI/THAER

APP:MasterCard
TVR:8040048000
CVR.420300
Terminal Type:22
TSI:6800
CID:80

9. PC POS to Service APIs

9.1 Messages from PC to POS service

All communications with the terminal is done through TCP IP messages. PC must get the terminal IP address and use the link to communicate with the terminal on port "7800".

The terminal must be in IDEAL mode (at screen home) when the PC calls it.

Please note that each request from the PC to the Terminal should start with this prefix ~PCNC~XXXX~

Also each response will start with ~PCNC~XXXX~

The XXXX is the length of the JSON request/response starting from **{** and ending with **}**.The brackets are included in the calculation of the length.

Messages exchange will be in a secure mode by exchanging **RSA keys size 4096** between PC and the terminal. The PC must create its own RSA keys, store the private component and call the INIT API to exchange the public components of the keys between the Terminal and the PC. The PC should store the Terminal public key received in the response to be able to use it during any next API call. The Idea of the INIT is to exchange the public component between the PC and the terminal. The PC when calling the INIT will send its public component in the request parameters, the terminal will reply with its own public key in the response.

This process (the call to the INIT) is required only once, the purpose is to exchange the keys between the 2 parties.

Security setup: PC generates its own RSA key with size 4096, then it prepares the INIT request and sends it to terminal. The terminal will respond with its public key component in the response. The PC should store it and use it on every next call as described below. The message request and response are described below.

Init Transaction

Request

Key Field	**Value data type**	**Max Size**	**Description**
`lang`	Integer	1 digit	Terminal interface and printout language; 0: English, 1: Arabic.
`type`	Alphanumeric	–	"**INIT**"
`rsaPubKey`	Alphanumeric	-	Client public RSA key with size 4096

Sample Requests:

Init sample:

~PCNC~1234~{"**rsaPubKey**":"MIICIjANBgkqhkiG9w0BAQEFAAOCAg8AMIICCgKCAgEAmjHgkOyOpTsQh14yGmgwY0W6LArhLBMHnq2NwvyVo48VZESGY2NfkJWZ4/4iMi/51g6GLF1CDGoFc2w8wVO40/4R3EVpFgxhqTl\u002BUXheMsJjzbRTkxe3DQTNpdSH8BCrQN/32BhDzginljT39mMeqm6wVxO6LkSV4bvqMJg2Gw\u

002BB\u002B2kn5eTt57jgyA7cEkEUSQC2h91bvfnzPJLFxVK5LyrVe8HTLRFETO/SisUhRuynaQ0LO4f2gCiaC\u002B5Fkv90vjI2UzD9vHki9a1OKhx3F4sk0\u002BhsDqUccsXjBE2K\u002BRtNdaT48nVxtmm9H401dfJTV5GRPLNUhDqS7T\u002B4\u002BP\u002BiMNW4AZ0MeMhZKlpJ96JDD3Ni79V/zqjtYLrlIi6OlS2kwmMFZLJsYRqqF9I2Ze80AIYejjluHhZYxHKlW9DmUvaXcOMdHjGVjNtqPSFEuZYdqLJYk2Lq61qrvPa9eqYzBMdxZd\u002Bs5uhQEzd5kxU2BziKKFSBAJtxCTCc\u002BcfR3iZCs11bSEn74d/2U\u002BgfAbRHpks0RPWO/i93FvIkyuwcTOni4TIzbpiIejXMCUf0gtz\u002BLTxU/FzgrJWNZMHfLQHKJSNdZ4JJ92XXIrUg\u002BiqmZpZ3C8nuqbzwn5vLk/hy7Use1TDFY5Yq9tODIO2Bw\u002B\u002B1wzZn8dNZw2fR8w\u002BSNVwh\u002BbECAwEAAQ==","**lang**":0,"**type**":"INIT"}

Response

Key Field	Value data type	Max Size	Description
`type`	Alphanumeric	Same as request	"**INIT**"
`resp_code`	Integer	4	0: success. Otherwise fail
`error_msg`	Alphanumeric	-	Error description
`rsaPubKey`	Alphanumeric	-	Client public RSA key with size 4096

Response Sample:

{"**resp_code**":0,"**rsaPubKey**":"MIICIjANBgkqhkiG9w0BAQEFAAOCAg8AMIICCgKCAgEAxFCAjOF1uMcSGJ1Fiiz5Bk2ztzLHwVDqaP9EgsJsc5y87DHPNKl7p0tWo16fBszisNB0zvCT0gNYyiiU6Bp8FrOMxF/XWe2ZOtJ7ud+yA+dWYaYPEf4R4MDgnvamTAmjFYRK+QzlsGqc0x/KTh4xKfXD0pWg+CUd337K4YQMLAiLNFZC2o5NTUHFxdSrTFq2flvxwt1tZDoXUbfe1aIYY8SooPOo13pBPNJranYoc4wk3d5Hfr+8Bfd2YeBT2PXcqMU34V6oxSXrMEhBABn85zanT+zz+wHTEL71k6ghsw/PPPZw14HygVdGB5LnkrzPMgpzFuvqEUNsSefi44sLpvjHmU7flIZS8WDEWotxeABje+TInSYYfKrfTHGDJ/qfuqvjlXBsp9g32qeK5z6Qijf9tOvfCSdWGWImgLBg8gEOPyffgffkKBhVrlOAbt0Pdvvv3KOC8bkWml4NaYcuAmSN5Ta7bltt24Lrb17LhulH+EMuSW4kfew2hGUcCwv0lWQeaPqskvf8ROefuEhlKfIuxmwIzB32izhFo2QrP3ovsRx5Y+MGgF+/xgLwSxgSTRl7Ogto92BKPF69jO1haCr/BJysXwjvfBtlRnCajhahZqeeJRAwFVfIGFpzrwrIL5GwdBIBzKuMmq27MmnYdpcpQmgnQnSDQB3/pSYNrXMCAwEAAQ\u003d\u003d","**type**":"INIT"}

Example on how to create the RSA keys on the PC using C sharpe.:

```
using RSACryptoServiceProvider provider = new(4096);
string priK = Convert.ToBase64String(provider.ExportEncryptedPkcs8PrivateKey(PcRsaPassword, new(
        PbeEncryptionAlgorithm.Aes256Cbc, HashAlgorithmName.SHA256, 1000)));
string pubK = Convert.ToBase64String(provider.ExportSubjectPublicKeyInfo());
```

Note: PC has to set its own password in field PcRsaPassword

After a successful INIT transaction, each request from the PC to the terminal should be encrypted by a **256-bits AES key**, this key will be encrypted using the Terminal Public RSA.

Each response from the terminal to the PC will be encrypted by a **256-bits AES key**, this key will be encrypted using the PC Public RSA.

The PC should generate a new **256-bits AES key** each time it sends a request to the terminal, and the same process will be done in the terminal when encryption process starts. The encrypted request and response should be in the below format:

{

“k”: “<the 256-bits AES key that encrypted by the RSA public key>”,

“d”: “<request/response body that encrypted by the 256-bits AES key>”

}

How to Handle the received encrypted data from the terminal:

1- When the encrypted response is received from the terminal, first it should decrypt the 256-bits AES **key field** using the PC Private RSA key component.
2- The result of step 1 is an unencrypted 256-bits AES key that should be used to decrypt the Data field.
3- Now the message response is in Plain format and can be used.

Important:

1. **RSA key should use PKCS1 padding when decrypt/encrypt.**
2. **AES should use ECB cipher mode and PKCS7 padding.**

Here is a sample for an encrypted sale request and response

1. ***Request Sample:***

Encrypted request:

{"**d**":"XJ2UtKFPa\u002BIEGoBGhOeiy/e2TQjqKhGJ2aM3JAD0aOIiDUXGhAabB3nPnrJbqQ8rW\u002B6lHJUQ6qgeqYaAYpUfR/2zYtwSg8eXvnBfJY1SOQamQF9PWORc\u002BI4CNsKwXscD","**k**":"IehR/\u002BqTiZyfCGVixKEl0egKt9OyJyfcN9RwviFvUVEe3miuc3F8USKBv4rVOk6QE\u002B5zEzqmWd2vBugCUi1D9keez2dXCv2JewEv2JDW1rx4O5akdxTVnJ5P5uVJj4vYbkgqQMLDl1XR42RgXZyoVnqkaMZzOx1qwhwVVcxJIBm8PwDLGKyYRQQZyCp9Ks5OSqxl55z02v03pLzYiHIZj\u002B5fWgZr0stgp25nErScb6nHNr/IL7R7eLX/prcFmJV0SJzrBIN6SvfA5lipR8sbPf0BeOvaMFLdKQn9y8\u002BJ9xIaxd62OMs\u002BauzvSZytiXAxwO8godDzYvyit\u002Bj5aaXEIxWw8rGu67g7PoI2BPsUBwpq5dAor5UCtkr6XvDWHH3a68z5cQqfr7h3wKQ6ryJthdiTEf/ir1IX92I0ywlkWXxBpHweLaK84dfQD9591Wro8QjFwtU8g\u002B9BEYGw4sQoNgCE2oE\u002BodQhK0JxWSqOeq\u002BoeNk1madPbVj0tztxDaTkEIYqe3zH/pp2eCCuAgEWrqAKdaq\u002BO1MS6NPEqIh0mnCMqZ0fLgGhdT5fIwoOVzB6LUNGxiDvreRVyTvMgxW7l0baNqysIiMFfuui2CrhzlLP9d1l3rB\u002Bnaf6n/R\u002BAPrSWUhxTO\u002BFOm\u002BQlaS9AT/5Xa3g5zdsEPDkDEYnjUEHSDk="}

Decrypted request:

{"cshbk":0,"rcpt":"ABC-123","cshr":0,"amt":4500,"cur":376,"lang":0,"type":"SALE","print":3}

2. ***Response Sample:***

Encrypted response:

{"**d**":"OnUBihGxTEuxcP8oZHnXkV70E+om8m54QZzoLZYoSC6aM+MMmspi0i4U08v2iOmkZuJPMIOCRgO3
X3gy8F40IwpNLb8Ujz7x5H7/QUNc9zRLd46NwZTPdy31p2DWwQ7kxNmbcAWfjbzkf9Kvc/x+YMMUIEG4d
o/SZmWokcKszY2FrlCutP3C0XN1S0iiQyFCPt4oTAPWfX79C6iUNQwVIUdHTDfGoG0VBaW2cHZR6yZQrnjd
u+F0KwYGgG+CNIKoWFeu/2mQALKhoHKOUyHMCvhPQWzONK9GW0/eNC8Kynj1iig0eRZ7rKcPXhhlQXfh
73V8PISBC7xyAXVITpthDzKcmo853cnMvxBVzZ9OmJuCM7qbru3jOTEVan1vakW/3vDHuzR/jM1HQHRDA
SCxlGEaDIqL1Sb1zm4Qk/ITmaRO0qX7Q4XklbPxAoNtHHCHiyr5+pw+qGDOKDk3AUkxRpFbiQoDTeCq6wcr
LRe2TBuuBWv+dK23hQhAOpzmjC+pTJ0X7I+4DGsHpGxzWAMm+7qK90MzqpM5mWt9NKqtYrTqG7yzbY/
7Ttm1AjgsFK/qddRnc2V5VY/zorQkdRiMX9po9r9sEns+QDdOvHDd12OQsOBgtkwVaNB7gXUXMKLRSCnA/
XW9cpbnLVU2B7b8BMMWTteKbqhSjG3lBI9XHj3WkU95AWPCUyNj1KFIwq/3fdVLCSkUUAixeVc2i42JVyw
ZwO3PtqWSk7d3J/QwzmQEyqmLOe0u+0axCS0IhXnZIFFHjFPcKOryHtMlaRgO9/T7bC/j0BNu0PCQUb79p2
CPGJCrwB7PJq2ZxFsPXa3IxpGss1igt4CvZ7YbKqGQABZMk4iBHNwSZoBn3KPHKdpHsP4KR1efszAcIpnUIKY
+t4PFCZeCrIMH1AvyOPfNGg\u003d\u003d","**k**":"QsWJqofmRy8nsVQzexZWQ7WJWg/CUozMY98SJCXoh
86PI9jMl5D2TIv+s6cIzunAIAT1sWvdwcDFl0auUtbLDc5mITlLpr2zx2S6W0bJcF3kc0byuQKuu6IW20AqGslZ
+sp80bdyAvQR/FKsTS9QAaDER2DHO1wOrTbpHms8OoRUFSJbRiCP70ZlX6FsvGJ9RxSt4ietat9eLCTYVWDO
H6WrkhCFtLVBShvBSWaTC7123fvydZuh8UEMlUzXICXJp+ln9bB6Ge9U79ol7jK9D85kEJE77YXWk7jukjAvb
1IMGZH+sn1PMoKVsgX3y2mSMcv1GZJk6RrL3tPRw8MQ1tdU7epqGMfSZaUDD44oirioyQXfjVfkmjCVY7Y
5IFwN4ZxB2t6mLHRi7W04b0Fd2nhQNXrluKu/IM1jck/3pfJtzWLBrm1D0AAG01LJ5FjNtknIDGmKISmBx6O
3l1SgRcjiHP3Nv85UMN5D0fBdaAZdkmN3jvUCNE1uPpRdgFMG53wxdMSfsmG0rOsoDw6FhOffBJaH06N
m0C3tNbk82CGFkikDiPt4mknMuGH1bUMYoUw0InkdAtN/q4TTsRfmcwahfVO+iPcn8HiQWdLFqJ8LLIP+h
NOsrfm8fnuumHB3rWKDuGUDNSLpGD/0JlEA4UQCJqSdz31bmFXx47H+YAU\u003d"}

Decrypted Response:

{"app_ver":"38000001,X990_00V1E0188660,05.01.00D\nMar 26 2023
02:20:58","auth_code":"000000","batch":7,"card":{"holder":"/","verif":"","entry":"CTLS","pan":"****00
35","pna_seq":1,"type":"VISA"},"dcc":"","error_msg":"Issuer not found\n(code
912)","icc":{"aid":"A0000000031010","cid":"80","trm_type":"22","tsi":"0000","tvr":"0000000000"},"inst
lmt":0,"is_qr_pay":false,"mcc":"7298","merch":"","outlet":"380000001","pos_code":"38000001","recon
":{"amt":0,"cur":"376","cname":"ILS"},"resp_code":912,"sale":{"cshbk":0,"amt":4500,"cur":"376","cnam
e":"ILS"},"seq":198,"stan":636,"datim":"230326144122","amt":4500,"cshr":0,"cur":376,"rcpt":"ABC-
123","lang":0,"print":3,"type":"SALE"}

9.2 Data Message Format

To reduce the size of data passed to the terminal, we will be using JSON messages between the PC and the service. The PC must send one of the following messages to the service:

- **SALE** - Sale Transaction
- **SALE_CB** - Sale with Cash-back Transaction
- **LOAN** - Loan Transaction
- **VOID** - Void Transaction
- **RETURN** - Return / Refund Transaction

- **QUERY** - Query Transaction
- **BATCH_TIME –** Set Batch Time Transaction

The JSON string contains <key, value> pairs where the key if always a text string, while the value could be either string or a primitive type (integer or Boolean).

The following section specify the details of the fields contained in each message.

Notes:

- All keys are case sensitive and must be sent/received as indicated.
- All fields are mandatory unless otherwise indicated.
- Maximum message size is 2000 bytes.
- All times are in local time zone and not in UTC.

Sale, Sale with Cash-back, and Loan Transactions

Request

Key Field	Value data type	Max Size	Description
`lang`	Integer	1 digit	Terminal interface and printout language; 0: English, 1: Arabic.
`type`	Alphanumeric	–	“**SALE**” or “**SALE_CB**” “LOAN” transaction type
`rcpt`	Alphanumeric	10 characters	**Conditional**. Used as a tracking reference number for the sender application request. In **SALE** type. (must be unique)
`cshr`	Integer	9 digits	**Optional**. Sent from the POS software indicating the cashier ID in the POS software.
`amt`	Numeric Text	15 digits	Transaction amount including the decimal digits without any decimal point character. The number of decimal digits will be used based on the currency field
`cur`	Integer	3 digits	Currency of the transaction amount. Currently we support only: 376 (ILS) and 840 (USD).
`cshbk`	Numeric Text	12 digits	**Conditional**. Present in sale with cashback only. The value has the same format as amount.
`instlmt`	Integer	2 digits	**Conditional**. Present in installment sales only. Number of installments of a loan transaction.
`print`	Integer	2 digits	Indicates what slip to print on terminal 1: customer slip only 2: customer and merchant slip 3: none

Sample Requests:

Sale sample*:* {“lang”:0, "type": "SALE", "amt": 1295, "cur": 840, "rcpt": "ABC-123", "cshr": 7}

This represents a sale of amount US$ 12.95.

Sale with cash back sample: {“lang”:0, "type": "SALE_CB", "amt": 1590, "cur": 376, "cshbk": 500, "rcpt": "TL1-037250", "cshr": 819}

This represents a sale with cashback. The total amount (including the cashback) is ILS 15.90; i.e., 10.90 + 5.00 ILS.

Loan sample: {"type": "LOAN", "amt": 60000, "cur": 376, "*instlmt": 12,* "rcpt": "TL1-037250", "cshr": 819}

This represents a loan transaction paid on 12 equal installments. The amount is ILS 600.00.

Response

Key Field	Value data type	Max Size	Description
`lang`	Integer	Same as request	0: English, 1: Arabic . The PINPAD display language
`type`	Alphanumeric	Same as request	“**SALE**” or “**SALE_CB**” LOAN transaction type
`print`	Integer	Same as request	Same as request
`rcpt`	Alphanumeric	Same as request	Same as request
`cshr`	Integer	Same as request	Same as request
`is_qr_pay`	Boolean		True or False.
`qr_ref`	Alphanumeric	10 characters	**Conditional:** Present if is QR pay
`app_ver`	Alphanumeric	50 characters	The terminal ID, serial, model, version of running app, etc.
`pos_code`	Numeric Text	8 digits	POS Terminal ID
`outlet`	Numeric Text	15 digits	Merchant Outlet Number
`merch`	Alphanumeric	40 characters	Merchant name
`datim`	Numeric Text	12 digits	The local date and time of the transaction as **YYMMDDHHmmss**; e.g., 191025175236, meaning 2019/10/25 17:52:36 (24-hour format).
`batch`	Integer	4 digits	Batch sequence number of the terminal
`seq`	Integer	6 digits; from 1 to 999999	Sequence number of the transaction as generated by the terminal
`stan`	Integer	6 digits; from 1 to 999999	System trace audit number of the transaction as generated by the terminal
`instlmt`	Integer	Same as request	**Conditional**. Present in installment sales only. Same as request.
`auth_code`	Alphanumeric	6 characters	Authorization code of the transaction
`resp_code`	Integer	3 digits	Response code of the transaction. Value = 000 for success; other code (001 to 999) for failure.

Key Field	Value data type	Max Size	Description
`ref_txt`	Alphanumeric	10 characters	**Conditional**. Sent from the terminal as a tracking code for the transaction.
`error_msg`	Alphanumeric	-	Error description
`sale`			**Conditional**. If not QR pay. Sale details in main currency
`amt`	Numeric Text	15 digits	Sale amount (no decimal digits) (including cashback when any)
`cur`	Numeric Text	3 digits	Sale currency ISO value ISO; e.g., 840
`cname`	Alphanumeric	3 characters	Sale currency ISO name ISO; e.g., USD
`cshbk`	Numeric Text	15 digits	**Conditional**. Cashback amount of the sale; only for “SALE_CB” transaction.
`recon`			**Conditional**. If not QR pay. Sale details in reconciliation currency.
`amt`	Numeric Text	15 digits	Reconciliation amount (no decimal digits)
`cur`	Numeric Text	3 digits	Reconciliation currency ISO value ISO; e.g., 840
`cname`	Alphanumeric	3 characters	Reconciliation currency ISO name ISO; e.g., USD
`card`			**Conditional**. If not QR pay. Details of the card used for transaction
`type`	Alphanumeric	20 characters	Card type; e.g., VISA, DEBIT, etc.
`entry`	Alphanumeric	20 characters	How card was used; e.g., Swipe, ICC, CTLS (Contactless tap)
`pan`	Numeric Text	19 digits	Card (Primary Account) Number. Mask last 4digits
`pan_seq`	Integer	2 digits	Card number sequence.
`holder`	Alphanumeric	26 characters	Card holder name.
`verif`	Alphanumeric	20 characters	How card holder was verified; e.g., NONE, PIN, Signature, or both PIN & Signature.
`icc`			**Conditional**. Card ICC values when entry was ICC or CTLS and not QR pay
`aid`	Numeric Text	16 digits	Application identifier; e.g., A00000000301
`tvr`	Numeric Text	16 digits	Terminal verification code; e.g., 00080248000
`tsi`	Numeric Text	16 digits	Transaction Status Information
`cvm`	Numeric Text	16 digits	Card Verification Method Result; e.g., 420300
`cid`	Numeric Text	2 digits	Cryptogram Information Data; e.g., 40
`trm_type`	Numeric Text	2 digits	Terminal Type; e.g., 22
`dcc`	Alphanumeric	63 characters	**Conditional**. DCC data when exists (see below for DCC field encoding).
`trx_type`	Alphanumeric	4 digits	Original trans type description; sale, sale with cashback, void, etc.

Key Field	Value data type	Max Size	Description
`voided`	Boolean	True or False	Whether trans is voided or not

Dynamic Currency Conversion (DCC) data field format

When a sale transaction contains currency conversion, the returned field is a positional field with the following information:

From Position	To Position	Length	Component Meaning
01	09	9	Exchange Rate
10	13	4	Exchange Rate Date (DDMM)
14	17	4	Cardholder Currency
18	29	12	Cardholder Amount
30	30	1	Currency Exponent
31	34	4	Conversion Margin Rate
35	37	3	Cardholder Currency Alpha
38	63	26	Cardholder Currency Name

Sample data: {…, "dcc": "1454.50002510036014981350   20300IDRRUPIAH   "}

Interpretation:

Data	Meaning	
`"1454.5000"`	Exchange rate:	**1.0 USD = 1454.5000 IDR**
`"2510"`	Exchange rate date:	**25/October**
`"0360"`	Currency ISO code:	**360**
`"14981350    "`	Amount (equivalent of USD 100 + 3%):	**149,813.50 IDR**
`"2"`	Currency exponent:	**2 decimal digits**
`"0300"`	Conversion margin:	**3.00 %**
`"IDR"`	Currency ISO short name:	**IDR**
`"RUPIAH                    "`	Currency ISO long name:	**RUPIAH**

Void and RETURN Transactions

Request

Key Field	Value data type	Max Size	Description
`type`	Alphanumeric	–	"**VOID**" or "**RETURN**" transaction type
`lang`	Integer	1 digit	Terminal interface and printout language; 0: English, 1: Arabic.
`amt`	Numeric Text	15 digits	Sale amount (no decimal digits) (including cashback when any)
`cur`	Numeric Text	3 digits	Sale currency ISO value ISO; e.g., 840
origSeq	Integer	6 digits; from 1 to 999999	Sequence number of the original transaction.
datim	Numeric Text	12	Original sale trans date time in format **yyMMddHHmmss** 24-hours format

Key Field	Value data type	Max Size	Description
authCode	Numeric Text	-	Original sale trans auth code.

Sample Request:

{“lang”:0, "type": "VOID", "amt": "2590", "cur": 376, "origSeq*": 1234,* "rcpt": "TL1-037250", "cshr": 819, “datim”: “241223152012”, “authCode”: “123456”}

The above represents a void sale for the transaction with sequence number 1234. The amount is ILS 25.90 (including the cashback) at 2024-12-23 15:20:12.

Response

The response will be the same information as that of the sale transaction with the “type” field same as the request value.

Query Transaction

This transaction can be used to retrieve the details of a previously executed transaction. The original transaction’s STAN (“origSeq”) must be known to retrieve the details.

Request

Key Field	Value data type	Max Size	Description
`type`	Alphanumeric	–	“**QUERY**” transaction type
`lang`	Integer	1 digit	Terminal interface and printout language; 0: English, 1: Arabic.
`origSeq`	Integer	6 digits	Number original transaction sequence returned from the terminal to the POS software
`rcpt`	Alphanumeric	10 characters	**Optional**. Used as a tracking reference number for the sender application request.
`cshr`	Integer	9 digits	**Optional**. Sent from the POS software indicating the cashier ID in the POS software.
`isQr`	Boolean	True or False	Indicates trans to search (QR or Card)

Sample Request:

{“lang”:0, "type": "QUERY", "origSeq*": 1234*}

The above represents a query for the transaction with sequence number 1234.

Response

Key Field	Value data type	Max Size	Description
`type`	Alphanumeric	Same as request	Same as request
`origSeq`	Integer	0	Dummy value
`found`	Boolean	True/false	Whether found the transaction or not
`trans`	Object	null / other	Transaction details. Same as original response

Key Field	Value data type	Max Size	Description
error_msg	Alphanumeric	-	Error description

Note: When transaction is found in the terminal, the "trans" value of the response will be similar to the response of the original transaction type; i.e., for "SALE" transaction the object will be the same as the response described in the SALE Transaction section above.

Batch Time Transaction

This transaction is to set a custom batch time.

Request

Key Field	Value data type	Max Size	Description
type	Alphanumeric	–	"**BATCH_TIME**" transaction type
lang	Integer	1 digit	Terminal interface and printout language; 0: English, 1: Arabic.
time	Alphanumeric	5	Batch time in format HH:mm Sample: 19:30

Sample Request:

{"lang":0, "type": "BATCH_TIME", "time"*: "19:30"*}

The above request will ask the terminal to set the batch time at 07:30 PM.

Response

Key Field	Value data type	Max Size	Description
resp_code	Integer	-	0 success otherwise fail
error_msg	Alphanumeric	-	Error description

10. Response Codes Returned from the Terminal

The following are the possible error code that can be returned in the status code for the various transactions.

POS Codes

Code	Description
0	Accepted
Any other value	Fail check the field error_msg for the exact error.